Deep Learning for Economics and Finance

Deep surrogates, GPs, Bayesian active learning, deep kernels, GP-VFI, active subspaces

Simon Scheidegger

HEC, University of Lausanne

University of Geneva
April 20–23 & May 4–7, 2026

Based on: Chen, Didisheim & Scheidegger (2026), *J. Financial Economics*; Friedl et al. (2023); Kübler, Scheidegger & Surbek (2026, forthcoming in *JPE Macroeconomics*)

Materials: `https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models`

Outline

Deep Surrogate Models

Gaussian Process Regression

Bayesian Active Learning

GPs for Dynamic Programming

Summary & Outlook

Motivation: Why Surrogates?

The computational bottleneck

Economic & financial models are expensive to evaluate:

- Solving Bellman equations / dynamic programming (DEQNs)
- PDE-based models (HJB, Black–Scholes for exotics)
- Monte Carlo simulations for risk measures

Key insight

We know the true model — we can generate *unlimited* training data by solving the model on a design of experiments.

Goal: Build a surrogate $\phi(\cdot \mid \theta_{NN})$ that approximates $f(\cdot)$ but evaluates *orders of magnitude faster*.

Look-up Tables $\rightarrow$ Neural Networks

Traditional: pre-compute & interpolate.

x	$\Phi(x)$
-2.0	0.0228
-1.0	0.1587
0.0	0.5000
1.0	0.8413
2.0	0.9772

Example: standard normal CDF table.

21st-century look-up table:

A neural network memorizes $\boldsymbol{x} \mapsto y$:

$$\phi(\boldsymbol{x}_t \mid \theta_{NN}) \approx y_t = f(\boldsymbol{x}_t)$$

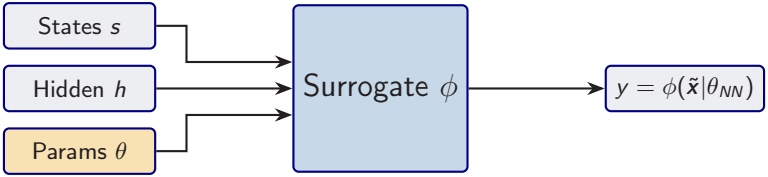
Continuous, differentiable, works in any dimension.

Pseudo-States: The Key Idea

Core insight (Chen, Didisheim & Scheidegger 2026)

Treat model parameters θ as extra state variables:

$$\tilde{\boldsymbol{x}} = \big(\underbrace{s_1, \ldots, s_n}_{\text{states}},\ \underbrace{h_1, \ldots, h_h}_{\text{hidden}},\ \underbrace{\theta_1, \ldots, \theta_p}_{\text{parameters}}\big) \in \mathbb{R}^d, \quad d = n + h + p.$$



Solve the model once over the full augmented space $\Rightarrow$ instant evaluation for *any* parameter configuration.

Building the Surrogate

1. **Design:** Draw $\tilde{\boldsymbol{x}}_i$ uniformly from $[\underline{x}, \bar{x}]^d, \quad i = 1, \ldots, m$.
2. **Label:** Compute $y_i = f(\tilde{\boldsymbol{x}}_i)$ using the expensive model.
3. **Train:** Minimize the empirical risk:

$$\hat{\theta}_{NN} = \arg\min_{\theta_{NN}} \frac{1}{m} \sum_{i=1}^{m} \left| \phi(\tilde{\boldsymbol{x}}_i \mid \theta_{NN}) - y_i \right|^2.$$

4. **Validate:** On a held-out test set, compute max / mean absolute error.

Remark

Since we can generate *unlimited* data from the true model, the training set can be made as large as needed $\Rightarrow$ no overfitting risk in the traditional sense.

Why Surrogates $\neq$ Standard ML

Standard supervised learning:

- Data is *scarce* and noisy
- Overfitting is a major concern
- Regularization, early stopping, cross-validation
- Small learning rates, careful tuning

Surrogate modeling:

- Data is unlimited (we own the model)
- Labels are exact (or noise-free)
- Can use large epochs, aggressive LR
- Focus on *approximation error*, not generalization

Once trained

- Highly accurate (errors $< 10^{-4}$ relative)
- Orders of magnitude faster than the original model
- Differentiable: gradients via automatic differentiation (backprop)

Speed Comparison: Option Pricing

From Chen, Didisheim & Scheidegger (2026), *J. Financial Economics*:

Task	**FFT**	**Surr. (CPU)**	**Surr. (GPU)**
Single price	23 ms	0.03 ms	—
1-day calib. (6×10^5)	3.3 h	18 s	0.4 s
1-year calib. (1.5×10^8)	$\approx$100 d	1.2 h	98 s

Takeaway

Deep surrogates enable real-time calibration of option pricing models — tasks that previously took days or were infeasible.

Comparison of Approximation Methods

Criterion	Polynomials/Splines	Sparse Grids	GPs	DNNs
High dimension ($d > 10$)	✗	∼	✗	✓
Local features / kinks	✗	✓	∼	✓
Irregular domains	✗	✗	✓	✓
Large training set ($N > 10^4$)	✓	∼	✗	✓
Built-in uncertainty	✗	✗	✓	✗

- **DNNs** excel in high dimensions with large data.
- **GPs** provide built-in uncertainty quantification — crucial for active learning and Bayesian inference.
- **Combining both** (GP for moderate d with UQ, DNN for high d) is a powerful strategy.

Application I: Structural Estimation

Goal: Estimate structural parameters θ from data via SMM:

$$\hat{\theta}_{SMM} = \arg\min_{\theta} \, [m(\theta) - \hat{m}^{\text{data}}]^\top \, W \, [m(\theta) - \hat{m}^{\text{data}}],$$

where $m(\theta)$ are model-implied moments and computing them naively requires solving and simulating the model for each candidate θ.

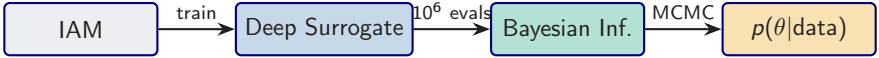
Surrogate-based estimation

1. Train a surrogate policy $\phi(x, \theta)$ over states and parameters (pseudo-states).
2. Reuse the trained surrogate to simulate moments $m(\theta)$ without re-solving the model.
3. Evaluate the SMM criterion over many θ values via grid search or standard optimization.

Result: Repeated SMM evaluations in seconds instead of weeks.
(Chen, Didisheim & Scheidegger 2026)

Application II: Uncertainty Quantification

DeepUQ (Friedl, Kübler, Scheidegger & Usui 2023):



- Surrogate over 100+ climate and economic parameters
- Posterior distributions of damage functions and optimal carbon tax
- MCMC requires millions of model evaluations, only feasible with a surrogate

Application III: Optimal Policy

Kübler, Scheidegger & Surbek (2025):

- High-dimensional integrated assessment model (IAM)
- GP surrogate + Bayesian Active Learning (BAL) for the value function
- GP provides *uncertainty quantification* at each point
- BAL selects training points where uncertainty is highest
- Result: optimal carbon tax policy with rigorous error bounds

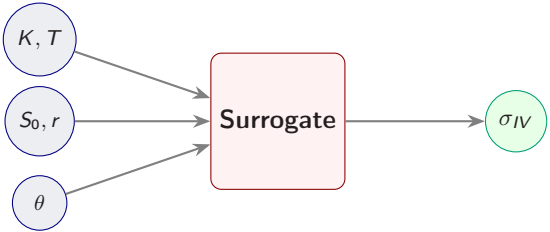
Preview

Parts III and IV of this lecture introduce Gaussian Processes and Bayesian Active Learning — the tools behind this application.

Application IV: The Implied Volatility Surface

Goal: Learn the mapping from option characteristics to implied volatility.

$$\sigma_{IV} = \phi(K, T, S_0, r, \theta_{\text{model}})$$



Benefits:

- **Calibration:** Invert the surrogate to find θ_{mod} matching market prices.
- **Arbitrage-free:** Can enforce constraints (no calendar/butterfly arb) in the loss.
- **Speed:** Enables real-time pricing for complex stochastic volatility models.

Pseudo-States: Concrete Use Cases

1. Exploring parameter distributions

Surrogate $\phi(s, \theta)$ maps *any* parameter θ to the model output instantly:

- Draw $\theta^{(j)} \sim p(\theta)$ from a prior or posterior distribution
- Evaluate $\phi(s, \theta^{(j)})$ for each draw — no re-solving
- Obtain the full distribution of outcomes: $\{y^{(j)}\}_{j=1}^{M}$

$\Rightarrow$ Bayesian inference, sensitivity analysis, robust policy design

Example: Climate–economy models

- Parameters: climate sensitivity, damage curvature, discount rate
- Surrogate over 100+ parameters trained once ($\sim$hours)
- MCMC posterior with 10^6 evaluations ($\sim$minutes with surrogate)

Pseudo-States: Policy Parameterization

2. Parameterizing policies

Include policy parameters θ_{pol} as pseudo-states. Optimize via autograd:

$$\theta^*_{\text{pol}} = \arg\max_{\theta_{\text{pol}}} \phi\big(s, \theta_{\text{model}}, \theta_{\text{pol}}\big)$$

Carbon tax

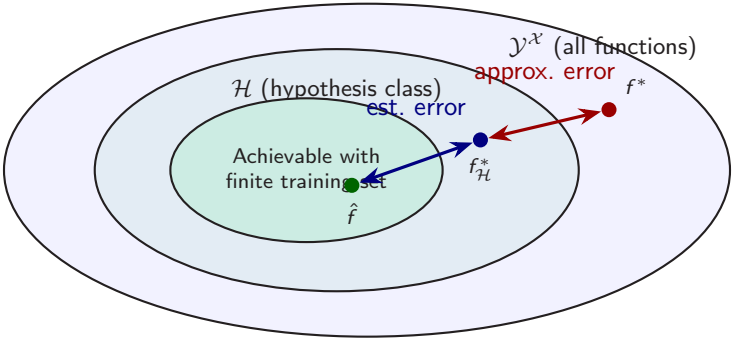
- Policy: tax trajectory θ_{pol}
- Model: climate sensitivity θ_{model}
- Optimal tax for each θ_{model}

Monetary policy

- Policy: Taylor rule coefficients
- Model: preference / technology shocks
- Optimal rule via gradient descent

Key: evaluate welfare for *any* model + policy configuration instantly $\Rightarrow$ no re-solving.

Sources of Approximation Error



- **Approximation error:** f^* may not lie in $\mathcal{H}$ $\Rightarrow$ increase network capacity.
- **Estimation error:** finite data $\Rightarrow$ mitigated by generating more training points.
- For surrogates, estimation error $\rightarrow 0$ as we can always generate more data.

Practical Tips for Surrogate Training

Architecture

- Swish activation ($x \cdot \sigma(x)$)
- 3–5 hidden layers, 128–512 neurons
- Linear output (no activation)

Training

- $m \geq 10^5$ samples (cheap to generate)
- Adam, LR 10^{-3}, reduce on plateau
- Hundreds of epochs (no overfit risk)

Validation

- Held-out test set
- Max / mean absolute error, R^2
- Scatter: prediction vs. truth

Loss functions

- MSE: $\frac{1}{m}\sum_i(\phi_i - y_i)^2$
- Relative L^1: $\frac{1}{m}\sum_i|(\phi_i - y_i)/y_i|$

Summary: Deep Surrogate Models

1. **Surrogates** = cheap, accurate replicas of expensive models.
2. **Pseudo-states** (θ as extra inputs) unlock:
   - Structural estimation (GMM in seconds)
   - Uncertainty quantification (MCMC with surrogate)
   - Optimal policy design
3. **Key advantage:** unlimited training data $\Rightarrow$ no overfitting.
4. **Speed:** orders of magnitude faster than original model.

Hands-on

Notebook: `01_Surrogate_Primer.ipynb` — Black–Scholes surrogate with implied volatility inversion.

Why Gaussian Processes?

- **Nonparametric:** No fixed architecture; the model grows with data.
- **Built-in uncertainty quantification:** Posterior mean *and* variance at every point.
- **Principled kernel selection:** Encode prior beliefs about smoothness, periodicity, etc.
- **Hyperparameter learning:** Marginal likelihood provides a principled objective.

Complementary to DNNs

- GPs: best for moderate dimensions ($d \leq 10$) with built-in UQ.
- DNNs: best for high dimensions ($d \gg 10$) with large data.
- Combining both is a powerful strategy (cf. slide 9).

Foundations: Rasmussen & Williams (2006); MacKay (1992); Krause, Singh & Guestrin (2008). **In DP:** Engel, Mannor & Meir (2005); Deisenroth, Rasmussen & Peters (2009). **Sparse / scalable:** Titsias (2009); Hensman, Fusi & Lawrence (2013).

Recall: Multivariate Gaussians

A random vector $\boldsymbol{x} \in \mathbb{R}^d$ is multivariate Gaussian if

$$\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$$

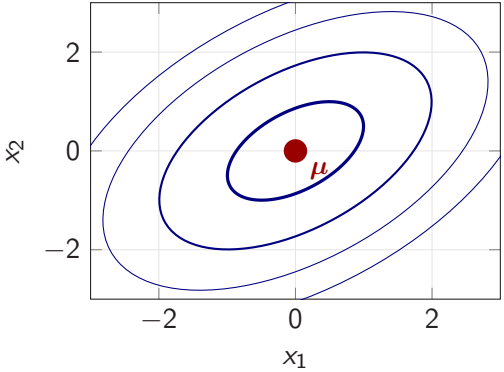
with density

$$p(\boldsymbol{x}) \propto \exp\Big(-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\Big)$$

Key fact

A multivariate Gaussian is fully characterized by its mean vector $\boldsymbol{\mu}$ and covariance matrix Σ.

2D Gaussian contours ($\rho = 0.5$)



Joint and Conditional Distributions

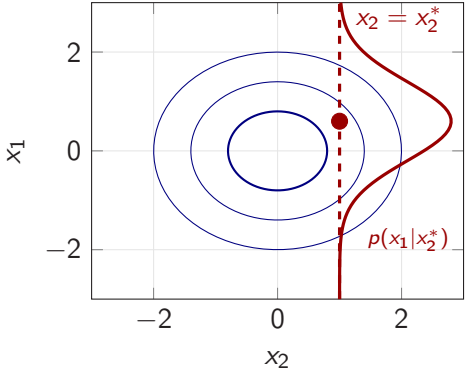
Joint distribution:

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{pmatrix} \right)$$

Conditional distribution:

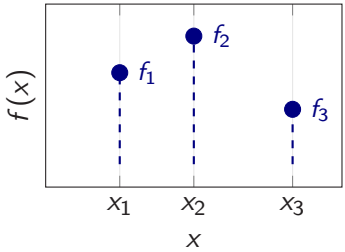
$$\mu_{1|2} = \mu_1 + \Sigma_{12}\Sigma_{22}^{-1}(x_2 - \mu_2)$$
$$\Sigma_{1|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21}$$

This is the mathematical engine behind GP regression.



"Slicing" the joint at $x_2 = x_2^*$ yields a 1D Gaussian.

From Observations to Interpolation



Assume function values are jointly Gaussian:

$$\begin{pmatrix} f_1 \\ f_2 \\ f_3 \end{pmatrix} \sim \mathcal{N}\left(0, \begin{pmatrix} K_{11} & K_{12} & K_{13} \\ K_{21} & K_{22} & K_{23} \\ K_{31} & K_{32} & K_{33} \end{pmatrix}\right)$$

Nearby points (f_1, f_2) should be more correlated than distant ones (f_1, f_3).

Example: $K = \begin{pmatrix} 1.0 & 0.7 & 0.2 \\ 0.7 & 1.0 & 0.6 \\ 0.2 & 0.6 & 1.0 \end{pmatrix}$

Covariance via a kernel function: $K_{ij} = k(x_i, x_j)$

$\Rightarrow$ The kernel encodes our prior belief about function smoothness.

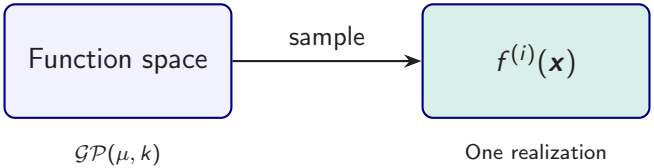
GP: A Distribution Over Functions

Definition

A Gaussian Process is a collection of random variables, any finite number of which have a joint Gaussian distribution:

$$f(\boldsymbol{x}) \sim \mathcal{GP}\big(\mu(\boldsymbol{x}),\ k(\boldsymbol{x}, \boldsymbol{x}')\big).$$

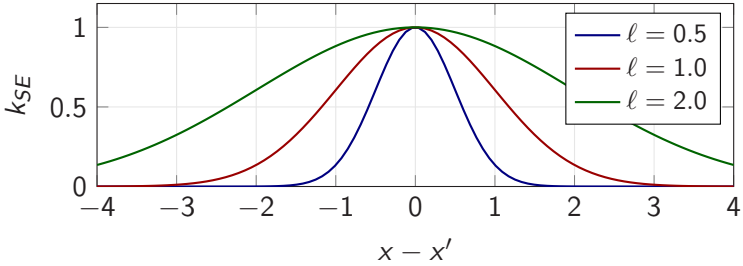
- **Mean function:** $\mu(\boldsymbol{x}) = \mathbb{E}[f(\boldsymbol{x})]$ (often set to 0).
- **Covariance function (kernel):** $k(\boldsymbol{x}, \boldsymbol{x}') = \mathbb{E}\big[(f(\boldsymbol{x}) - \mu(\boldsymbol{x}))(f(\boldsymbol{x}') - \mu(\boldsymbol{x}'))^\top\big]$.



The Squared Exponential (RBF) Kernel

$$k_{SE}(x, x') = \sigma_f^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

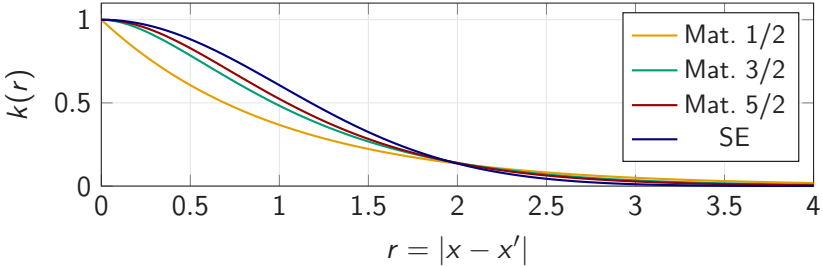
- ℓ = **length scale** (horizontal correlation range)
- σ_f^2 = **signal variance** (vertical scale)



Larger $\ell \Rightarrow$ smoother functions; smaller $\ell \Rightarrow$ more wiggly.

The Matérn Kernel Family

$$k_{\text{Mat}}(r) = \sigma^2 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}\, r}{\ell}\right)^{\nu} K_{\nu}\left(\frac{\sqrt{2\nu}\, r}{\ell}\right), \quad r = |x - x'|$$

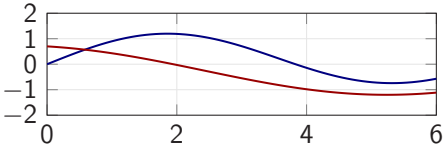


- ν controls smoothness: $\nu = 3/2 \Rightarrow$ once differentiable; $\nu = 5/2 \Rightarrow$ twice; SE $\Rightarrow$ ∞-smooth.
- **Use case:** Matérn when the true function is *not* infinitely smooth (e.g., has kinks).

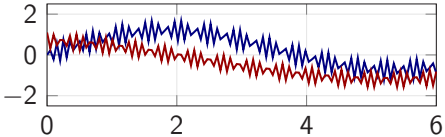
Combining Kernels

Kernels can be added and multiplied to compose complex priors from simple blocks.

SE kernel (smooth trend)



SE + Periodic (trend + seasonality)



Kernel algebra

- $k_1 + k_2$ (OR):
  high if *either* is high
  $\Rightarrow$ trend + seasonality
- $k_1 \times k_2$ (AND):
  high only if *both* high
  $\Rightarrow$ locally periodic

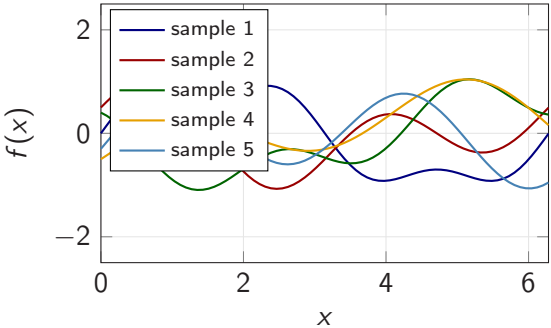
Compose complex priors from simple building blocks.

Sampling from the GP Prior

Algorithm

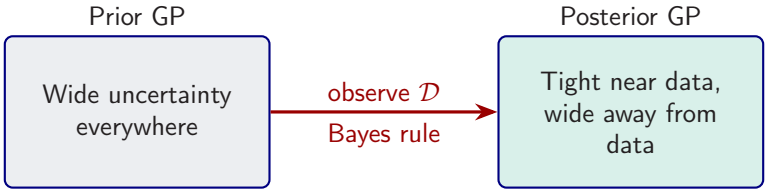
1. Grid $x_{1:N}$ over domain
2. $K_{ij} = k(x_i, x_j)$
3. Cholesky: $K = LL^\top$
4. $\boldsymbol{f}^{(s)} = L \cdot \boldsymbol{u}$, $\boldsymbol{u} \sim \mathcal{N}(0, I)$

Five functions drawn from a GP prior with SE kernel ($\ell = 1$, $\sigma_f = 1$).



From Prior to Posterior: Bayes Rule

- **Training set:** $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$.
- **Test points:** $\boldsymbol{X}_*$ where we want predictions.
- Joint distribution of training outputs $\boldsymbol{f}$ and test outputs $\boldsymbol{f}_*$ is Gaussian.



The posterior is *still a GP* — fully characterized by updated mean and covariance.

Noiseless GP Regression

Joint distribution:

$$\begin{pmatrix} \boldsymbol{f} \\ \boldsymbol{f}_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu} \\ \boldsymbol{\mu}_* \end{pmatrix}, \begin{pmatrix} K & K_* \\ K_*^\top & K_{**} \end{pmatrix} \right)$$

where $K_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_j)$, $(K_*)_{ij} = k(\boldsymbol{x}_i, \boldsymbol{x}_*^{(j)})$, $(K_{**})_{ij} = k(\boldsymbol{x}_*^{(i)}, \boldsymbol{x}_*^{(j)})$.

Posterior (conditioning on $\boldsymbol{f}$):

$$\boldsymbol{\mu}_{*|\mathcal{D}} = K_*^\top K^{-1}(\boldsymbol{f} - \boldsymbol{\mu}) + \boldsymbol{\mu}_* \tag{1}$$

$$\Sigma_{*|\mathcal{D}} = K_{**} - K_*^\top K^{-1} K_* \tag{2}$$

Key insight

- Near observed data: $K_*^\top K^{-1} K_* \approx K_{**} \Rightarrow$ small variance.
- Far from data: $K_*^\top K^{-1} K_* \approx 0 \Rightarrow$ large variance (reverts to prior).

Prediction at a Single Test Point

Noiseless setting ($\sigma_y^2 \to 0$ limit; GP interpolates training data). For a single test input $\boldsymbol{x}_*$:

$$\bar{f}_* = \boldsymbol{k}_*^\top K^{-1} \boldsymbol{f} = \sum_{i=1}^{N} \alpha_i \, k(\boldsymbol{x}_i, \boldsymbol{x}_*) \tag{3}$$

$$\sigma_*^2 = k(\boldsymbol{x}_*, \boldsymbol{x}_*) - \boldsymbol{k}_*^\top K^{-1} \boldsymbol{k}_* \tag{4}$$

where $\boldsymbol{\alpha} = K^{-1}\boldsymbol{f}$, $\boldsymbol{k}_* = (k(\boldsymbol{x}_1, \boldsymbol{x}_*), \ldots, k(\boldsymbol{x}_N, \boldsymbol{x}_*))^\top$. Noisy form ($K \to K + \sigma_y^2 I$) on next slide.

Interpretation

Posterior mean is a weighted sum of kernel basis functions centered at training points:

$$\bar{f}_* = \sum_{i=1}^{N} \alpha_i \, k(\boldsymbol{x}_i, \boldsymbol{x}_*)$$

Each observation $\boldsymbol{x}_i$ "pulls" the prediction via its kernel similarity to $\boldsymbol{x}_*$.

GP Regression with Noisy Observations

Observation model:

$$y = f(\boldsymbol{x}) + \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \sigma_y^2)$$

The covariance of *observations* becomes:

$$\mathrm{cov}[y_p, y_q] = k(\boldsymbol{x}_p, \boldsymbol{x}_q) + \sigma_y^2 \delta_{pq}$$

In matrix form: replace K with

$$K_y = K + \sigma_y^2 I$$

Effect of noise

- Predictions no longer interpolate the data exactly.
- Even at training points, there is residual uncertainty.
- The noise term $\sigma_y^2 I$ also acts as numerical regularization ("nugget" / "jitter").

Noisy GPR: Posterior Equations

Posterior mean and variance with noise:

$$\bar{f}_* = \boldsymbol{k}_*^\top (K + \sigma_y^2 I)^{-1} \boldsymbol{y} \tag{5}$$

$$\sigma_*^2 = k(\boldsymbol{x}_*, \boldsymbol{x}_*) - \boldsymbol{k}_*^\top (K + \sigma_y^2 I)^{-1} \boldsymbol{k}_* \tag{6}$$

Noiseless ($\sigma_y^2 = 0$):

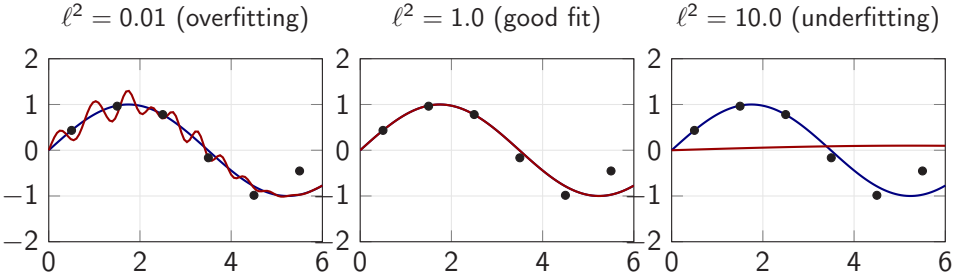
- Interpolates training points exactly
- Variance $= 0$ at training inputs
- Requires K to be positive definite

Noisy ($\sigma_y^2 > 0$):

- Smooths through training points
- Variance > 0 everywhere
- Better numerical conditioning

In practice: Even for noiseless surrogates, adding a small jitter $\sigma_y^2 \sim 10^{-6}$ improves numerical stability.

The Effect of Kernel Parameters



- **Blue:** true function $f(x) = \sin(0.9x)$.
- **Red:** GP posterior mean.
- **Too small** ℓ**:** overfits noise/wiggles. **Too large** ℓ**:** underfits, misses structure.

$\Rightarrow$ We need a principled way to learn ℓ and σ_f^2.

Learning Kernel Hyperparameters

Problem: How to choose ℓ, σ_f, σ_y? $\Rightarrow$ Maximize the marginal likelihood.

Marginal likelihood (model evidence)

$$\log p(\boldsymbol{y} \mid \boldsymbol{X}) = \underbrace{-\tfrac{1}{2}\boldsymbol{y}^\top K_y^{-1}\boldsymbol{y}}_{\text{data fit}} \underbrace{-\tfrac{1}{2}\log|K_y|}_{\text{complexity penalty}} \underbrace{-\tfrac{N}{2}\log(2\pi)}_{\text{constant}}$$

Intuition:

- **Data fit** $\uparrow$: reward explaining observations
- **Complexity** $\downarrow$: penalize overly flexible models
- Balance = automatic Occam's razor

Optimization:

- Gradient ascent on $\log p(\boldsymbol{y}|\boldsymbol{X})$
- Hyperparameters: $\theta = \{\ell, \sigma_f, \sigma_y\}$
- Closed-form gradients available
- Implemented in scikit-learn, GPyTorch

Leave-One-Out Predictive Error

Idea. Quantify out-of-sample predictive accuracy without holding out data.

For a GP with training set $\{(\boldsymbol{x}_i, y_i)\}_{i=1}^n$ and noise variance σ_y^2:

$$\mu_{-i}(\boldsymbol{x}_i) - y_i \;=\; -\frac{[K_y^{-1}\boldsymbol{y}]_i}{[K_y^{-1}]_{ii}}, \qquad \sigma_{-i}^2(\boldsymbol{x}_i) \;=\; \frac{1}{[K_y^{-1}]_{ii}}.$$

Why it's free.

- K_y^{-1} is recovered from the same Cholesky factor used for posterior inference
- LOO across all n points: $\mathcal{O}(n^2)$ on top of the existing $\mathcal{O}(n^3)$
- No new training, no held-out split lost

Use cases. Surrogate-health diagnostic across iterations of GP-VFI (Part IV) and across active-learning rounds in SMM (companion exercise deck).

Closed form: Rasmussen & Williams (2006), §5.4.2.

GPR: Numpy Implementation Sketch

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

def kernel(X1, X2, l=1.0, sf=1.0):
    sq = np.sum(X1**2,1).reshape(-1,1) \
         + np.sum(X2**2,1) - 2*X1@X2.T
    return sf**2 * np.exp(-0.5 * sq / l**2)

K = kernel(X_tr, X_tr) + 1e-8*np.eye(N)
L, low = cho_factor(K, lower=True)  # Cholesky
alpha = cho_solve((L, low), y_tr)   # weights

Ks = kernel(X_tr, X_te)
mu  = Ks.T @ alpha                  # mean
v   = cho_solve((L, low), Ks)
var = kernel(X_te, X_te) - Ks.T @ v # variance
```

See **Algorithm 2.1** in Rasmussen & Williams (2006), and notebook `02_GP_and_BAL.ipynb`.

GP in Practice: scikit-learn & GPyTorch

scikit-learn:

- `GaussianProcessRegressor`
- Automatic hyperparameter tuning via marginal likelihood
- Great for prototyping ($N \leq 10^4$)
- CPU only

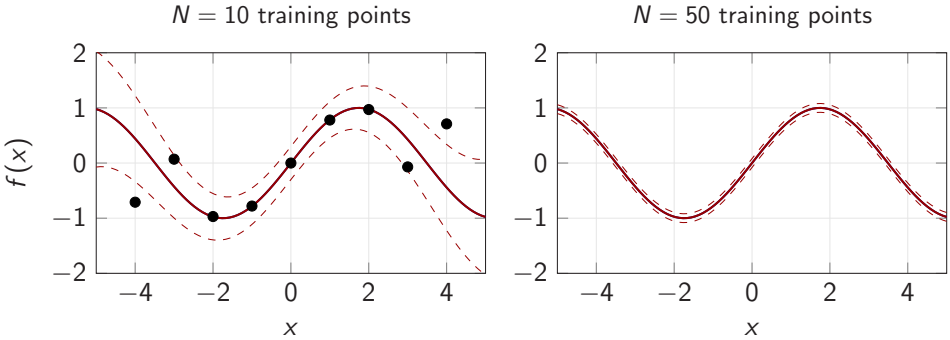
GPyTorch:

- GPU-accelerated GP inference
- Scalable to $N > 10^5$ (inducing points, KISS-GP)
- Integrates with PyTorch ecosystem
- Production-ready

Hands-on

Notebook: `02_GP_and_BAL.ipynb` — GP regression from scratch, scikit-learn, and Bayesian Active Learning.

Prediction & Confidence Intervals



Blue: true function. **Red:** posterior mean. **Dashed:** 95% confidence interval.

More data $\Rightarrow$ tighter confidence intervals.

Algorithmic Complexity

Cost of GP inference

- **Training:** Cholesky decomposition of $K_y \in \mathbb{R}^{N \times N}$: $O(N^3)$.
- **Prediction (per test point):** $O(N)$ for mean, $O(N^2)$ for variance.
- **Hyperparameter optimization:** $O(N^3)$ per gradient step.

Scaling strategies for large N

- **Sparse GPs:** Inducing point methods — approximate K with rank-M matrix ($M \ll N$), cost $O(NM^2)$.
- **Random features:** Approximate kernel with random Fourier features.
- **GPyTorch:** KISS-GP, structured kernel interpolation, GPU acceleration.

For the surrogate problems in this course ($N \leq 10^4$, $d \leq 10$), standard GPs are sufficient.

GPs vs DNNs: When to Use Which

Criterion	Gaussian Processes	Deep Neural Networks
Uncertainty quantification	Built-in (posterior variance)	Requires ensembles / MC dropout
Hyperparameter tuning	Marginal likelihood	Grid search / Bayesian opt.
Scaling with N	$O(N^3)$, limit $\sim 10^4$	$O(N)$ per epoch
Scaling with d	Curse of dimensionality	Handles $d \gg 100$
Small data ($N < 100$)	Excellent	Typically underfits
Training	Closed-form	SGD, many epochs
Interpretability	Kernel encodes prior	Black box

Rule of thumb

- $d \leq 10$, **need UQ:** GPs (+ BAL).
- $d > 10$, **large data:** DNNs.

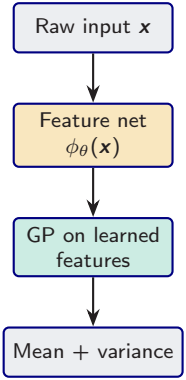
Deep Kernel Learning: Neural Features + GP Uncertainty

Idea: learn a feature map $\phi_\theta(\boldsymbol{x})$ with a neural network, then use

$$k_{\mathrm{DKL}}(\boldsymbol{x}, \boldsymbol{x}') = k_{\mathrm{base}}\big(\phi_\theta(\boldsymbol{x}), \phi_\theta(\boldsymbol{x}')\big)$$

so the GP operates in the learned feature space.

- **Front end:** the DNN learns a geometry where the target is smoother.
- **Back end:** the GP still provides posterior mean and variance.
- **Training:** optimize the feature map and kernel hyperparameters via the GP marginal likelihood.



When Do Deep Kernels Help?

Standard GP assumption: Euclidean distance in the *raw input space* is informative.

Failure modes:

- Hidden regimes / thresholds / nonstationarity
- Inputs are badly warped coordinates of a simple latent state
- Moderate-to-high-dimensional raw features with strong correlations
- You still need uncertainty quantification or BAL

Rule of thumb

- If the input is already low-dimensional and smooth: **plain GP** is usually enough.
- If the latent structure is smooth but the raw geometry is poor: **DKL** is often the right bridge.
- If d is huge and UQ is not central: **pure DNN surrogate**.

In short: DKL answers the question "Can we combine DNN feature learning with GP uncertainty?"

Summary: Gaussian Process Regression

1. **GPs** = nonparametric Bayesian regression with built-in UQ.
2. **Kernel** encodes prior beliefs (smoothness, scale, periodicity).
3. **Posterior:** closed-form mean and variance, conditioned on data.
4. **Hyperparameters:** learned by maximizing the marginal likelihood (automatic Occam's razor).
5. **Limitation:** $O(N^3)$ scaling; best for moderate N and d.
6. **DKL:** neural feature extractor + GP head = useful when raw inputs are warped or moderately high-dimensional, but you still want UQ.

Next: Use the GP's uncertainty to intelligently select training points — Bayesian Active Learning.

Motivation: Where to Sample?

Uniform sampling:

- Places points evenly across the domain
- Wastes evaluations in smooth regions
- Cannot adapt to function complexity

Adaptive / active sampling:

- Concentrates points where the function is hard to approximate
- Uses the model's uncertainty to guide selection
- Fewer evaluations for the same accuracy

Key question

Given a budget of N model evaluations, which input points $\{x_1, \ldots, x_N\}$ minimize the overall approximation error?

Bayesian Active Learning: The Idea

Use the GP posterior variance to guide sampling.

Acquisition / score function

$$U(\boldsymbol{x}) = w_{\text{exp}} \cdot \mu(\boldsymbol{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\boldsymbol{x})$$

- $\mu(\boldsymbol{x})$: posterior mean (exploitation — sample where function is large)
- $\sigma^2(\boldsymbol{x})$: posterior variance (exploration — sample where uncertainty is high)
- $w_{\text{exp}}, w_{\text{var}} \geq 0$: trade-off weights (distinct from the AR(1) coefficient ρ and the discount factor β used elsewhere).

Pure exploration ($w_{\text{exp}} = 0$): maximize $\sigma^2(\boldsymbol{x})$, i.e., uncertainty sampling.

Mixed ($w_{\text{exp}} > 0, w_{\text{var}} > 0$): balance exploration and exploitation.

For surrogate building, pure exploration (max variance) is often sufficient.

BAL Algorithm

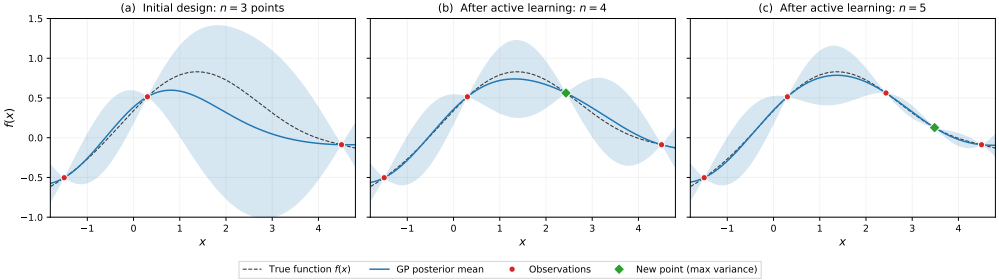
Algorithm 1 Bayesian Active Learning for Surrogates

Require: Initial design $\mathcal{D}_0 = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{n_0}$, budget N, candidate set $\mathcal{C}$

Fit GP on $\mathcal{D}_0$: compute posterior mean $\mu(\cdot)$ and variance $\sigma^2(\cdot)$
for $t = 1, \ldots, N - n_0$ **do**
    Compute score $U(\boldsymbol{x}) = w_{\text{exp}}\, \mu(\boldsymbol{x}) + \frac{w_{\text{var}}}{2} \log \sigma^2(\boldsymbol{x})$ for all $\boldsymbol{x} \in \mathcal{C}$
    Select $\boldsymbol{x}_{\text{new}} = \arg\max_{\boldsymbol{x} \in \mathcal{C}} U(\boldsymbol{x})$
    Query the expensive model: $y_{\text{new}} = f(\boldsymbol{x}_{\text{new}})$
    Update: $\mathcal{D}_t \leftarrow \mathcal{D}_{t-1} \cup \{(\boldsymbol{x}_{\text{new}}, y_{\text{new}})\}$
    Refit GP on $\mathcal{D}_t$
end for
return Trained GP surrogate

BAL in Action: Progressive Refinement

BAL selects each new point where the posterior variance is largest (green diamond), causing the credible band to shrink exactly where it was widest.



(a) 3 initial observations (red dots); GP posterior (blue) deviates in data-sparse regions with wide 95% CI.

(b) Acquisition function picks max-variance point (green diamond); posterior tightens locally.

(c) Second iteration fills the remaining gap — 5 points suffice to track the true function.

Key contrast with uniform: BAL concentrates points where the function is hardest to approximate.

BAL in Economics

Grid-free approximation

BAL + GP = grid-free adaptive sparse grids:

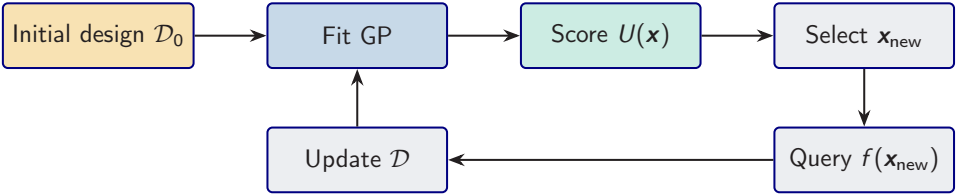
- No structured grid needed
- Refinement via posterior variance
- Scales to $d \leq 10$

Applications

- **Optimal carbon tax**
- **Dynamic incentives** (kinks)
- **Asset pricing** (moderate d)

References: Renner & Scheidegger (2018); Kübler, Scheidegger & Surbek (2025).

Combining BAL and GP for Surrogates



Why this combination works

- **GP provides UQ:** posterior variance tells us where the approximation is uncertain.
- **BAL selects training points:** maximize information gain per model evaluation.
- **Surrogate = trained GP:** the same GP used for selection serves as the surrogate.

Ideal for $d \leq 10$ when each model evaluation costs minutes to hours.

Summary: Bayesian Active Learning

1. **BAL** = principled data selection using GP uncertainty.
2. **Score function** balances exploration (high σ^2) and exploitation (high μ).
3. **Faster convergence** than uniform sampling, especially for functions with kinks or local features.
4. **Application:** surrogate construction for moderate-dimensional problems.

Hands-on

Notebook: `02_GP_and_BAL.ipynb` — GP from scratch, scikit-learn GP, and BAL comparison.

Part IV

Gaussian Processes for Dynamic Programming

Based on: Scheidegger & Bilionis (2019), *J. Comput. Sci.* 33, 68–82.

What is Dynamic Programming?

Dynamic programming (DP) is the core method for solving sequential decision problems under uncertainty—the backbone of macro, finance, and structural IO.

The idea: An agent makes decisions over time to maximize total discounted utility:

$$V(\boldsymbol{x}_0) = \max \sum_{t=0}^{\infty} \beta^t \, r(\boldsymbol{x}_t, \xi_t)$$

subject to $\boldsymbol{x}_{t+1} \sim F(\cdot \mid \boldsymbol{x}_t, \xi_t)$.

Bellman's principle of optimality:

$$V(\boldsymbol{x}) = \max_{\xi} \left\{ r(\boldsymbol{x}, \xi) + \beta \, \mathbb{E}[V(\boldsymbol{x}_{t+1})] \right\}$$

Solve this one equation instead of the infinite-horizon problem.

Why DP matters:

- Asset pricing, consumption–savings
- Growth theory, capital accumulation
- Labor: job search, human capital
- Climate: optimal carbon taxation

Challenge: Bellman equation on a high-dimensional state space—the curse of dimensionality.

Example: The Stochastic Optimal Growth Model

Workhorse testbed for computational macro (cf. Days 2–3). Model with D sectors:

- State: $\boldsymbol{k} = (k_1, \ldots, k_D)$
- Controls: consumption $\boldsymbol{c}$, labor $\boldsymbol{l}$, investment $\boldsymbol{i}$
- Production: $f(k_j, l_j) = A k_j^{\psi} l_j^{1-\psi}$
- Shocks: $\epsilon_{t,j} \sim \mathcal{N}(0, \sigma^2)$
- $k_j^+ = (1-\delta)k_j + i_j$

D ranges from 1 (textbook) to 500.

Bellman equation:

$$V(\boldsymbol{k}) = \max_{\boldsymbol{c},\boldsymbol{l}} \left\{ u(\boldsymbol{c}, \boldsymbol{l}) + \beta\, \mathbb{E}\left[V(\boldsymbol{k}^+) \right] \right\}$$

Challenge: approximate V on $\mathbb{R}^D$ at each VFI step. Grid methods fail for $D > 20$. GPs as interpolation $\Rightarrow$ arbitrary geometries, scales to 500D.

Hands-on

`04_GP_Value_Function_Iteration.ipynb` — solves the $D = 1$ case with GP-based VFI.

The Key Idea: GP as Value Function Surrogate

At each VFI step s:

1. Generate n^s training inputs $\boldsymbol{X}$ in the state space
2. Evaluate Bellman: $t_i^s = (TV^{s-1})(\boldsymbol{x}^{(i)})$
3. Learn GP surrogate V_{surr} from $\{\boldsymbol{X}, \boldsymbol{t}\}$
4. Set $V^s = V_{\text{surr}}$ (posterior mean)
5. Error: $\eta^s = \|V^s - V^{s-1}\|_\infty / \Delta_V$

GP posterior mean $\rightarrow$ next iterate; GP posterior variance $\rightarrow$ free uncertainty bounds at every step.

Why GPs instead of grids?

- Work on arbitrary geometries—no tensor-product structure
- Training points adaptively chosen—concentrate near steep gradients
- Built-in interpolation uncertainty
- Non-converging points can be discarded—impossible with grids

Recall

Same GP posterior from Part II, now applied iteratively inside a DP algorithm.

Origin: GPDP — Deisenroth, Rasmussen & Peters (2009, *Neurocomputing*); economics/high-dim extension — Scheidegger & Bilionis (2019, *JoCS*).

Algorithm: ASGP Value Function Iteration

Input: Initial guess $V_{\mathbf{next}}$, tolerance $\bar{\eta}$
while $\eta > \bar{\eta}$ **do**
  Generate n^s training inputs $\boldsymbol{X} = \{\boldsymbol{x}^{(i)}\} \in [\underline{\boldsymbol{k}}, \bar{\boldsymbol{k}}]^D$
  for $\boldsymbol{x}^{(i)} \in \boldsymbol{X}$ **do**
    Bellman target: $t_i = (TV_{\mathbf{next}})(\boldsymbol{x}^{(i)})$; policy: $\xi_j(\boldsymbol{x}^{(i)}) \in \arg\max TV(\boldsymbol{x}^{(i)})$
  end for
  Learn surrogate $V_{\mathbf{surr}}$ with ASGP (or GP) from $\{\boldsymbol{X}, \boldsymbol{t}\}$
  Error: $\eta = \|V_{\mathbf{surr}} - V_{\mathbf{next}}\|_\infty$; $V_{\mathbf{next}} \leftarrow V_{\mathbf{surr}}$
end while
Output: V, policy functions $\xi^* = (\xi^*_{\mathbf{1}}, \ldots, \xi^*_{\mathbf{3}D})$

Key: Training set $\boldsymbol{X}$ can be adaptively steered—non-converging points excluded, new points added. Impossible with grids.

Cheap BAL Inside the VFI Loop

Goal of acquisition. Uniform interpolation accuracy of V, not optimization $\Rightarrow$ pure exploration.

Acquisition function. Posterior standard deviation only:

$$\boldsymbol{x}^{\text{next}} \in \arg\max_{\boldsymbol{x} \in \mathcal{X}^{\text{cand}}} \sigma^{s}_{\text{GP}}(\boldsymbol{x}) \quad \text{(no UCB, no EI).}$$

Recipe.

- Every N VFI iterations, score a Latin-hypercube candidate set
- Pick top n_{add} by σ_{GP}, with min-spacing constraint to avoid clustering
- Evaluate Bellman operator at new points (parallel) and append to $\mathcal{D}^{s}$

Why pure exploration here. A UCB-style acquisition would chase high-value states; we instead want σ_{GP} small everywhere so the approximate Bellman operator stays close to a contraction.

Notebook 04 ($N = 12$, $n_{\text{add}} = 2$): active design beats naive LHS on both Bellman residual and LOO-RMSE.

Active Subspaces: Breaking the Curse of Dimensionality

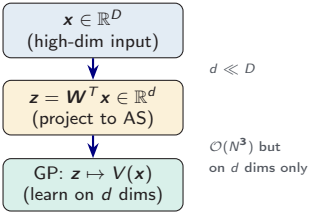
Standard GPs fail for $D \gtrsim 10$ (Euclidean distance uninformative). Active Subspaces (AS) find a low-dim manifold of maximal response variation.

Idea: $f(\boldsymbol{x}) \approx h(\boldsymbol{W}^T \boldsymbol{x})$, $\boldsymbol{W} \in \mathbb{R}^{D \times d}$, $d \ll D$.

Finding $\boldsymbol{W}$:

1. Collect gradients $\boldsymbol{g}^{(i)} = \nabla f(\boldsymbol{x}^{(i)})$
2. $\boldsymbol{C}_N = \frac{1}{N} \sum \boldsymbol{g}^{(i)} (\boldsymbol{g}^{(i)})^T$
3. SVD: $\boldsymbol{C}_N = \boldsymbol{V} \boldsymbol{\Lambda} \boldsymbol{V}^T$; find sharp drop in λ_i
4. $\boldsymbol{W} = [\boldsymbol{v}_1, \ldots, \boldsymbol{v}_d]$ (top d eigenvectors)

The ASGP pipeline:



In the growth model, AS dim is typically $d = 1$, even when $D = 500 \Rightarrow$ tractable.

Hands-on: `05_Active_Subspace_2D`, `06_..._10D`, `07_..._Nonlinear` — AS construction and ASGP vs GP comparison.

When the Active Manifold is Curved

Linear AS assumes the important directions are linear combinations of inputs: $f(\boldsymbol{x}) \approx h(\boldsymbol{W}^T\boldsymbol{x})$. If f depends on a nonlinear aggregate of its inputs, the linear projection has to carry every direction that enters the aggregate – not the aggregate itself.

Radial-ridge example ($D = 20$):

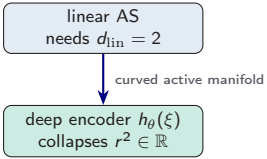
$$y(\xi) \;=\; \exp\Big(-\big[(w_1^\top \xi)^2 + (w_2^\top \xi)^2\big]\Big).$$

- $C = \mathbb{E}[[]\,\nabla y\,\nabla y^\top]$ has two nonzero eigenvalues.
- Linear AS must pick $d_{\mathrm{lin}} = 2$.
- Yet the true intrinsic coordinate is the scalar $r^2 = (w_1^\top \xi)^2 + (w_2^\top \xi)^2$ – one dimension.

Symptoms of curvature:

- Spectral gap ambiguous; λ_i drops slowly.
- ASGP accuracy plateaus early as d grows.
- Physics suggests thresholds, ridges, radial coords.

two linear features → one nonlinear aggregate



Fix: replace the linear projection $\boldsymbol{W}^T\boldsymbol{x}$ by a learned nonlinear encoder and jointly train a link function (Tripathy & Bilionis, 2018).

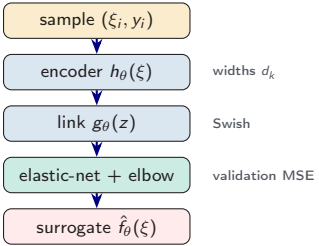
Hands-on: `09_Deep_Active_Subspace_Ridge` – validates that deep AS recovers $d_{\mathrm{nl}} = 1$ on the radial ridge.

Deep Active Subspaces (Tripathy & Bilionis, 2018)

Parametrize the surrogate as $\hat{f}(\xi) = g\big(h_\theta(\xi)\big)$, with encoder $h_\theta : \mathbb{R}^D \to \mathbb{R}^d$ and link $g_\theta : \mathbb{R}^d \to \mathbb{R}$ both small MLPs. Setting $h_\theta(\xi) = U_m^\top \xi$ recovers the linear-AS ansatz, so the nonlinear encoder generalizes it.

Three design choices:

1. **Widths** decay exponentially $d_k = \lceil D \exp(\eta_{\mathrm{w}} k) \rceil$, $\eta_{\mathrm{w}} = \frac{1}{L} \log(d/D)$.
2. **Swish activation** $\sigma(z) = z/(1 + e^{-z})$ (smooth, non-saturating).
3. **Elastic-net loss** $\mathcal{L} = \frac{1}{N} \sum (\hat{f} - y_i)^2 + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2$.



Gradient-free. Learned from (ξ_i, y_i) alone – no ∇f samples, no orthogonality constraint.

Choosing d. No spectral gap; use a validation-MSE elbow across $d = 1, 2, 3, \ldots$

Rule of thumb: fit linear AS first; escalate to deep AS when (i) no gradients, (ii) ambiguous spectral gap, or (iii) $D \gg 10$ with curved manifold. **Hands-on:** `09_..._Ridge` (curved wins), `10_..._Borehole` (linear wins).

Recipe: Fit a Deep Active Subspace

Self-contained recipe for the Tripathy-Bilionis architecture. All steps rely only on (ξ_i, y_i) pairs – no gradients, no Stiefel constraint, no kernel choice.

Algorithm.

1. **Sample** $(\xi_i, y_i)_{i=1}^N$; 80/20 train/val split.
2. **Standardize** inputs (unit variance), center outputs.
3. For $d \in \{1, 2, 3, \ldots\}$: encoder $h_\theta : \mathbb{R}^D \to \mathbb{R}^d$ with widths $d_k = \lceil D e^{\eta_w k} \rceil$, link $g_\theta : \mathbb{R}^d \to \mathbb{R}$ (16–32 units); train on $\mathcal{L} = \mathrm{MSE}(\hat{f}, y) + \lambda_1 \|\theta\|_1 + \lambda_2 \|\theta\|_2^2$ with Adam + cosine LR.
4. **Pick** d at the *validation-MSE elbow*.
5. **Deploy**: $\hat{f}_\theta$ is the surrogate; $h_\theta(\xi) \in \mathbb{R}^d$ is a nonlinear active coordinate usable downstream (GP, sensitivity, optimization).

Default hyperparameters:

encoder depth L	3
link hidden units	16–32
activation	Swish ($\gamma = 1$)
optimizer	Adam, lr 5×10^{-3}
schedule	cosine, 10^3–$2 \cdot 10^3$ eps
λ_1, λ_2	10^{-5}, 10^{-4}
batch size	full batch if $N \lesssim 10^4$

Samples: $N \approx 50\, d_{\mathrm{nl}}$ to find the bottleneck, $\approx 200\, d_{\mathrm{nl}}$ for deployment.

Sanity checks: (i) validation curve monotone-then-flat in d; (ii) scatter of $h_\theta(\xi_i)$ vs. the first linear-AS coordinate shows whether the encoder has actually gone *nonlinear*.

Reading the Validation-MSE Elbow

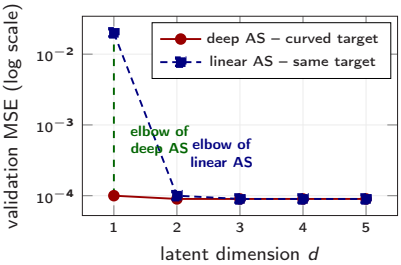
The spectral-gap heuristic of linear AS has no direct analogue for a neural encoder. Tripathy & Bilionis (2018) replace it by a validation-MSE elbow: train several latent dimensions and stop when extra capacity stops reducing held-out error.

How to read it.

- Plateau for $d \geq d_{\mathrm{nl}} \Rightarrow$ intrinsic dim reached.
- Linear AS needs $d_{\mathrm{lin}} \geq d_{\mathrm{nl}}$ whenever the aggregate is nonlinear: the elbow sits below the spectral gap.
- If curves cross (as on the borehole) the function is nearly a ridge and linear AS + polynomial link is more data-efficient.

Three common shapes:

- Sharp elbow at $d = 1$ – classic radial / ridge target; deep AS wins.
- Gradual decay – weak low-rank structure; more samples or richer encoder needed.
- Flat – no low-dim structure; return to the original D-dimensional problem.



Stylized picture of the radial-ridge experiment (notebook 09): deep AS flattens at $d = 1$, linear AS at $d = 2$.

Operational test: a drop of $< 2\times$ in validation MSE between d and $d+1$ is not worth paying the extra latent dimension for in most applications.

Parallelization: Embarrassingly Parallel VFI

Bellman evaluations at n^s training points are independent $\Rightarrow$ embarrassingly parallel.

MPI workflow:

1. **Broadcast** current V_{next} to all workers
2. **Evaluate:** each worker solves n^s/n_{cpu} Bellman problems
3. **Gather** training targets at master
4. **Fit GP** surrogate (done once, centrally)

Communication cost negligible. Most time in parallelizable Bellman evaluations.

Scaling (stochastic growth model):

D	Nodes	Total CPU h
10	1	0.01 h
50	10	0.5 h
100	20	4 h
200	50	30 h
500	100	$\sim$3700 h

CPU time grows sub-exponentially ($<$2 days wall-clock for 500D with 100 nodes).

Results: 1D to 500D Stochastic Growth

Key results (see Figs. 1–5 in the paper):

- **1D:** Convergence in $\sim$70 VFI iterations; error $10^{-1} \to 10^{-5}$; GP CI bands negligible with just 10 training points
- **10D:** GP alone works; ASGP ($d = 1$) equally accurate
- **50–500D:** ASGP converges to 10^{-4} normalized error; all dimensions converge in $\sim$70 iterations

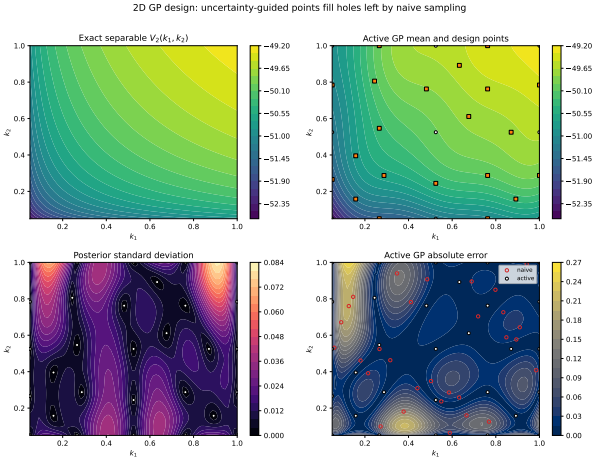
Why this works:

- Active Subspace dimension is $d = 1$ even for $D = 500 \Rightarrow$ GP operates on 1D projected input
- Training set adaptively steered (drop non-converging points)
- Grid methods (Smolyak) cannot handle $D > 100$; ASGP avoids the explicit grid-based curse (does not eliminate all high-d costs)

Hands-on

Notebook: `04_GP_Value_Function_Iteration.ipynb` — solves the 1D growth model with GP-based VFI using scikit-learn. Shows convergence, value function with CI bands, and policy functions.

2D GP-VFI in Action



Separable benchmark $V_2(k_1, k_2) = V^*(k_1) + V^*(k_2)$ with $\delta = 1$ closed form. Active design places points where σ_{GP} is largest. (Notebook 04, cell 25.)

PAUSE — 5 min Finger Exercise: One Step of GP-Based VFI

Exercise

Consider a 1D growth model with $V(k) = \max_c\{\log(c) + \beta\, V(k')\}$, $k' = k^{0.36} - c$, $\beta = 0.96$. Suppose you have a GP trained on 3 points: $V(0.5) = -8.0$, $V(1.5) = -2.5$, $V(2.5) = 0.5$.

1. At $k = 1.0$ (a new point not in the training set): the Bellman maximization yields $V^{\text{new}}(1.0) = -4.1$. How does the GP posterior update when you add this observation?
2. Does the GP posterior variance at $k = 1.0$ increase or decrease after adding this point?
3. Why is it advantageous that training points can be added or removed freely, unlike grid-based methods?

Solution: One Step of GP-Based VFI

Answer

1. The GP is retrained on 4 points: $\{0.5, 1.0, 1.5, 2.5\}$. The posterior mean $\mu(k)$ now interpolates through all 4 observations exactly (noiseless GP). The posterior mean between existing points adjusts to accommodate the new observation.
2. The posterior variance at $k = 1.0$ decreases to zero (the GP passes through the observation exactly). Variance at nearby points also decreases. Variance far from all training points remains large.
3. Flexibility: If the Bellman operator at some point fails to converge (e.g., near a constraint), that point can simply be dropped. On a grid, removing a point breaks the tensor-product structure. This is a key advantage of GP-based VFI over grid-based methods.

UQ as a Byproduct & Learning Ergodic Sets

Uncertainty quantification:
The GP posterior variance provides free UQ for the interpolated value function at every state point and VFI iteration. Policy uncertainty must be propagated through the maximization step.

Parameter uncertainty propagation:

- Add uncertain parameters as pseudo-states: $\tilde{S} = (\boldsymbol{k}, \gamma)$ where $\gamma \in [\underline{\gamma}, \bar{\gamma}]$
- Solve the model once on the extended state space
- Extract univariate effects: $\mathcal{M}_i(\chi_i) = \mathbb{E}[\mathcal{M}(\Theta) \mid \Theta_i = \chi_i]$
- Global sensitivity analysis (Sobol-style) in a single computation

This avoids the traditional approach of re-solving the model for each parameter value.

Learning ergodic sets:
Many economic models have irregularly shaped ergodic sets (ellipsoids, not hypercubes).

1. Solve model on a large domain initially
2. Simulate the economy to generate capital paths $\{\boldsymbol{k}_t^+\}$
3. Fit a Bayesian Gaussian Mixture Model to learn $\hat{\rho}_{\text{ergodic}}(\boldsymbol{x})$
4. Re-solve on the ergodic set only—sample training points from $\hat{\rho}_{\text{ergodic}}$ instead of the full hypercube

GPs handle this naturally: no grid structure to reconfigure. Training points can be placed anywhere in the state space.

GPs vs. DEQNs for Solving Dynamic Models

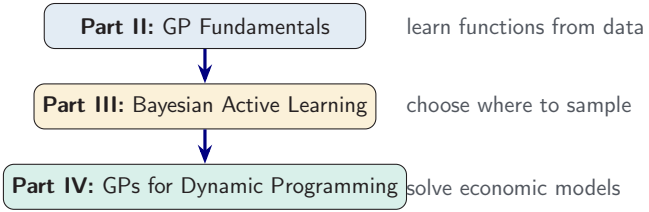
Both are complementary tools—choose based on problem structure:

	GP / ASGP	**DEQNs (Days 2–4)**
Method	Value function iteration	Euler equation residuals
Dimensionality	Up to ∼500 (with AS)	>1000 feasible
UQ built-in	**Value** (GP variance)	**No**
Hardware	CPU clusters (MPI)	GPUs
Irregular domains	**Yes**	**Yes**
Sensitivity analysis	Cheap with pseudo-states	Pseudo-states or re-training

Use GPs when effective dimension is moderate and you need value-function UQ or sensitivity analysis.
Use DEQNs when D is very large, you have GPU access, or you need complicated market-clearing conditions.

Summary: The Full GP Pipeline for Economics

The same GP machinery, applied at increasing levels of ambition:



Reference: Scheidegger & Bilionis (2019), "Machine Learning for High-Dimensional Dynamic Stochastic Economies," *J. Computational Science* 33, 68–82.

Hands-on

Notebook: `04_GP_Value_Function_Iteration.ipynb` (in `day7/code/`).

Recap: The Computational Toolbox

PINNs

- Continuous-time PDEs
- Physics loss
- Mesh-free
- HJB, Black–Scholes

Deep Surrogates

- Discrete-time
- Pseudo-states
- Fast evaluation
- Estimation, UQ

GPs + BAL

- Nonparametric
- Built-in UQ
- Active learning
- Moderate dim

ASGP + VFI

- GP inside DP
- Active Subspaces
- Up to 500D
- Ergodic sets

All four approaches are complementary: choose based on the problem structure, dimensionality, whether UQ is needed, and available hardware.

The Toolbox: When to Use What

Method	Best for	Key strength
DEQN	Discrete-time equilibrium models	Scales to high d, Euler eq. loss
PINN	Continuous-time PDEs/HJB	Mesh-free, physics-constrained
Deep Surrogate	Fast evaluation, estimation, UQ	Pseudo-states, autograd
GP + BAL	Moderate d, need UQ	Built-in uncertainty, active learning
Deep Kernel	Warped inputs + need UQ	Learned features + GP uncertainty

Decision heuristic

1. Is it a continuous-time PDE? $\Rightarrow$ **PINN**.
2. Do you need to re-evaluate for many parameter values? $\Rightarrow$ **Deep Surrogate**.
3. Is $d \leq 10$ and you need UQ / active learning? $\Rightarrow$ **GP + BAL**.
4. Is the latent structure smooth, but the raw input geometry is poor? $\Rightarrow$ **Deep Kernel**.
5. Is it a discrete-time equilibrium model? $\Rightarrow$ **DEQN**.

References & Resources

Key papers:

- Chen, Didisheim & Scheidegger (2026). *Deep Surrogates for Finance: With an Application to Option Pricing*. J. Financial Econ.
- Scheidegger & Bilionis (2019). *ML for High-Dim Dynamic Stochastic Economies*. J. Comput. Sci.
- Kübler, Scheidegger & Surbek (2026). *Using Machine Learning to Compute Constrained Optimal Carbon Tax Rules*. Forthcoming in *JPE Macroeconomics*.
- Rasmussen & Williams (2006). *GP for ML*. MIT Press.
- Wilson et al. (2016). *Deep Kernel Learning*. AISTATS.

Notebooks: `01_Surrogate_Primer`, `02_GP_and_BAL`, `03_Structural_Estimation_BM`, `04_GP_Value_Function_Iteration`, `05-07_Active_Subspace` (2D, 10D, nonlinear), `08_Deep_Kernel_Learning`, `09_Deep_Active_Subspace_Ridge`, `10_Deep_AS_vs_Linear_AS_Borehole`

GitHub: https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

Questions?